Documentation

IFT-2103 Minigolf TP1

Sommaire:

1. Définition de l'Environnement **2**
a. Vue d'ensemble 2
b. Environnement Technique 2
2. Diagramme de la Boucle de Jeu **3**
3. Définition de l'Action Réalisée **5**
a. Actions de Jeu Principales 5
Action 1 : Coup de Balle (Shot) Préconditions : 5
Action 2 : Mouvement Continu Préconditions : 5
Action 3 : Rebond (Bounce) Préconditions : 6
Action 4 : Arrêt de la Balle (Stop) Préconditions : 6
4. Formes de Collisionneurs Supportées **7**
a. Sphère (Sphère) 7
b. Box (Boîte Rectangulaire) 7
c. Cylindre (Cylinder) 7
d. Plan (Plane) 8
e. Matrice de Compatibilité des Collisions 8
5. Optimisation de la Détection de Collision 9
a. Continuous Collision Detection (CCD) 9
b. Triages et Optimisations 9
i. Tri par Proximité 9
ii. Séparation Statique/Dynamique 10
iii. Seuils de Vélocité 10
c. Complexité Théorique 10
6. Réactions aux Collisions 11
a. Pipeline de Réaction de Collision 11
b. Processus Détaillé 11
c. Cas Spéciaux de Collision 12
d. Paramètres de Collision Ajustables 13
e. Exemple de Séquence Complète 13
7. Architecture Modulaire 15
8. Conclusion 16

1. Définition de l'Environnement

a. Vue d'ensemble

Le projet **Minigolf TP1** est un jeu de minigolf développé avec le moteur **Unity**, implémentant un système de physique personnalisé et une détection de collisions avancée. Le projet utilise une architecture modulaire avec les composants suivants :

- **Système de physique personnalisé** (`CustomPhysics`) : Intégration numérique semi-implicite d'Euler
- **Détection de collisions continue** (`CollisionDetector`) : Détection prédictive avec CCD (Continuous Collision Detection)
- **Système de formes de collision** : Sphère, Box, Cylindre et Plan
- **Contrôleur de balle** (`BallController`) : Gestion de la boucle de jeu et des interactions

b. Environnement Technique

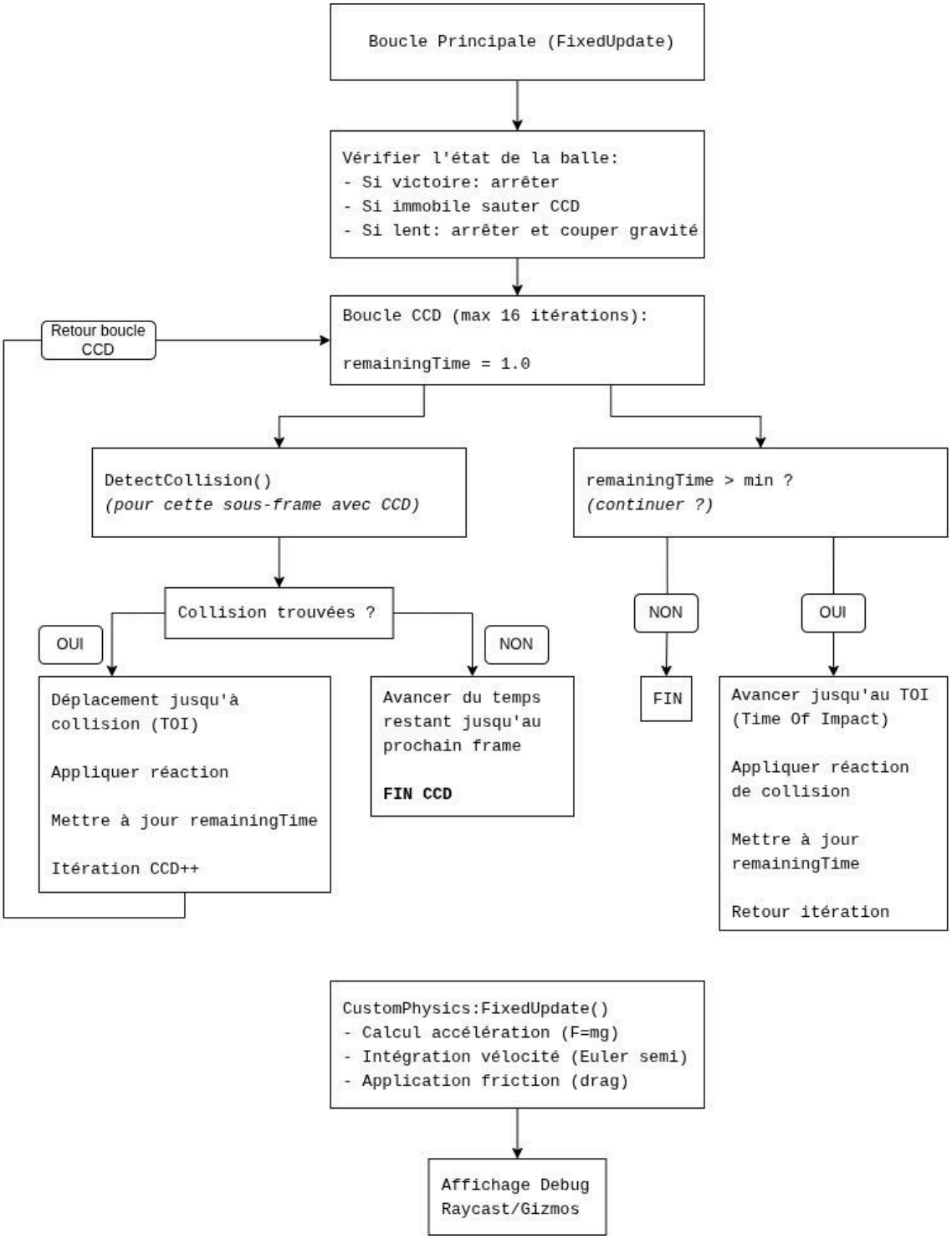
Composant	Détails
Moteur	Unity (version définie dans *ProjectSettings/ProjectVersion.txt)*
Langage	C#
Architecture	ECS-adjacent avec composants Unity
Système d'entrée	Input System (New Input System)
Physique	Moteur personnalisé basé sur des lois classiques

Configuration Physique de la Balle de Golf

```
Masse (m)                          : 0.04593 kg (balle de golf standard)
Gravité (g)                        : 9.81 m/s²
Coefficient de friction linéaire (c) : 0.5 s¹
Coefficient de restitution (e)     : 0.7 (rebonds partiels)
Seuil d'arrêt                      : 0.01 m/s
```

---

2. Diagramme de la Boucle de Jeu



Phases de la Boucle de Jeu

- **Initialisation** (Start)
  - Récupération des composants Physics et Collision
  - Enregistrement de la position de départ
  - Initialisation du CollisionDetector
- **Déplacement (FixedUpdate)**
  - Vérification de l'état de la balle
  - Exécution de la boucle CCD pour gérer les collisions multiples
  - Application des forces physiques
- **Résolution des Collisions**
  - Détection des collisions (statiques et dynamiques)
  - Calcul du TOI (Time Of Impact)
  - Tri par proximité
  - Application de la réaction physique

---

3. Définition de l'Action Réalisée

a. Actions de Jeu Principales

Action 1 : Coup de Balle (Shot) Préconditions :

- ➔ Balle immobile (vitesse ffi 0.01 m/s)
- ➔ Pas encore de victoire
- ➔ Joueur appuie sur un bouton d'action

Effets :

- ➔ Application d'une impulsion initiale à la balle
- ➔ Démarrage du mouvement physique
- ➔ Incrémentation du compteur de coups

Loi Physique Appliquée :

```
Δv = Impulse / masse

Exemple :
Force = hitPower = 2 N
Impulse = Force × Δt = 2 N × 0.02 s = 0.04 N·s
Δv = 0.04 / 0.04593  0.87 m/s
```

Action 2 : Mouvement Continu Préconditions :

- ➔ Vitesse > 0.01 m/s
- ➔ Pas de collision imminente

Effets :

- ➔ Mise à jour de la position selon la vélocité
- ➔ Application de la gravité
- ➔ Application de la friction

Lois Physiques Appliquées :

01. Deuxième Loi de Newton (F = ma)

```
a = ΣF / m
ΣF = mg (gravité) + résistance de l'air
```

02. Intégration Semi-Implicite d'Euler

```
a(t) = F(t) / m
v(t+Δt) = v(t) + a(t) × Δt
p(t+Δt) = p(t) + v(t+Δt) × Δt
```

Avantages : Stabilité meilleure que l'Euler explicite, peu coûteux en calcul

03. Friction Linéaire (Drag)

```
a_drag = -c × v            (où c = coefficient de friction = 0.5 s¹)
v(t+Δt) = v(t) + a_drag × Δt

Effet : ralentissement exponentiel
v(t) = v × e^(-c×t)
```

Action 3 : Rebond (Bounce) Préconditions :

➔ Collision détectée avec une surface
➔ Vitesse > vitesse minimale pour rebondir (0.2 m/s)

Effets :

➔ Inversion de la composante de vélocité selon la normale
➔ Perte d'énergie selon le coefficient de restitution
➔ Incrémentation du compteur de rebonds

Loi Physique Appliquée :

Réflexion avec Restitution (Coefficient of Restitution)

```
Réflexion parfaite : v' = v - 2(v·n)n
Avec perte d'énergie : v_final = v' × e

Où :
v = vélocité avant collision
n = normale de la surface (normalisée)
e = coefficient de restitution (0.7 pour balle de golf)

Exemple :
v = (1, 0, 0) m/s (horizontal)
n = (0, 1, 0) (normal vertical vers le haut)
v·n = 0
v' = (1, 0, 0) - 2(0)(0, 1, 0) = (1, 0, 0) // Aucune composante
   verticale
```

Action 4 : Arrêt de la Balle (Stop) Préconditions :

➔ Vitesse < 0.01 m/s

Effets :

➔ Définition de la vélocité à zéro
➔ Désactivation de la gravité
➔ Marque d'une itération comme complète

4. Formes de Collisionneurs Supportées

Le système supporte 4 types de formes de collision :

a. Sphère (Sphère)

- **Type** : SphereCollisionShape
- **Propriétés** : Rayon (radius)
- **Utilisée pour** : Balle de golf
- **Tests disponibles** :
  - Sphère vs Sphère
  - Sphère vs Box
  - Sphère vs Cylindre
  - Sphère vs Plan

Équation mathématique :

```
Centre : C = transform.position
Rayon : r
Point sur sphère : P tel que |P - C| = r
```

b. Box (Boîte Rectangulaire)

- **Type** : BoxCollisionShape
- **Propriétés** : Dimensions (width, height, depth)
- **Utilisée pour** : Obstacles, parois
- **Tests disponibles** :
  - Box vs Sphère
  - Box vs Box (dans les développements futurs)

Équation mathématique :

```
Coins (6 faces) définis par position et demi-dimensions AABB (Axis-Aligned
Bounding Box) ou OBB (Oriented Bounding Box)
```

c. Cylindre (Cylinder)

- **Type** : CylinderCollisionShape
- **Propriétés** : Rayon (radius), Hauteur (height)
- **Utilisée pour** : Colonnes, trous
- **Tests disponibles** :
  - Cylindre vs Sphère
  - Cylindre vs Cylindre

Équation mathématique :

```
Centre : C
Rayon : r
Hauteur : h
Point sur surface : x² + z² = r² (projection XZ) y  [C.y - h/2, C.y + h/2]
```

d. Plan (Plane)

- **Type** : PlaneCollisionShape
- **Propriétés** : Normal (Vector3), Distance (D)
- **Utilisée pour** : Terrain, surfaces planes
- **Tests disponibles** :
  - Plan vs Sphère

Équation mathématique :

```
Équation du plan : N · P = D

Où :
N = normale du plan (normalisée)
P = point dans l'espace
D = N · P (où P est un point sur le plan)

Distance d'un point M au plan :
d = |N · M - D|
```

e. Matrice de Compatibilité des Collisions

	Sphère	Box	Cylindre	Plan
Sphere	✅	✅	✅	✅
Box	✅	❌	❌	❌
Cylindre	✅	❌	✅	❌
Plan	✅	❌	❌	❌

---

5. Optimisation de la Détection de Collision

a. Continuous Collision Detection (CCD)

La balle se déplace potentiellement très vite et pourrait traverser des obstacles. Le système utilise une **détection prédictive** pour éviter ce problème.

Seuil d'activation :

```
velocity² > PREDICTIVE_VELOCITY_THRESHOLD_SQ = 0.0001
Soit v > 0.01 m/s
```

Algorithme CCD :

1. Prédiction de Trajectory
   - ➔ Projection du mouvement de la balle sur le frame suivant
   - ➔ Calcul du Time Of Impact (TOI) avec chaque obstacle
2. Itération de Sous-Cycles

Pour chaque itération (max 16) :

- Déterminer le temps jusqu'à la collision (TOI)
- Déplacer la balle jusqu'au point de collision
- Appliquer la réaction de collision
- Réduire le temps restant
- Continuer si temps_restant > 0.001s

3. Formule du TOI pour Sphère vs Plan

```
TOI = (d - N·C) / (N·v)

Où :
d = distance du plan
N = normale du plan
C = centre de la sphère
v = vélocité de la sphère

Si TOI < 0 : déjà en collision
Si TOI > deltaTime : pas de collision ce frame
```

b. Triages et Optimisations

i. Tri par Proximité

```
// Après détection, trier par TOI (collision la plus proche en premier)

collisions.Sort((a, b) => a.timeOfImpact.CompareTo(b.timeOfImpact));
```

Bénéfice : Traite d'abord la collision la plus imminente, plus réaliste 7

ii. Séparation Statique/Dynamique

```
List<CollisionShape> dynamicShapes; // Objets mobiles
List<CollisionShape> staticShapes; // Terrain, obstacles fixes
```

Bénéfice : Réduit les tests inutiles (balle vs balle, par exemple)

iii. Seuils de Vélocité

- **Seuil d'arrêt** : 0.01 m/s
- **Seuil CCD** : 0.01 m/s (√0.0001)
- **Seuil d'itération minimale** : 0.001s de temps restant

Bénéfice : Évite les calculs pour les objets quasiment immobiles

c. Complexité Théorique

Opération	Complexité	Notes
Détection de collisions par frame	O(n·m)	n = objets statiques, m=objets dynamiques
CCD (pire cas)	O(n·m·16)	16 itérations max
Tri par TOI	O(k log k)	k=collisions détectées (généralement petit)

Optimisation Future :
Implémentation d'une **Quadtree** ou **Spatial Parti tioning** pour réduire O(n·m) à O(log n)

---

6. Réactions aux Collisions

a. Pipeline de Réaction de Collision

```
Détection →          Résolution Position →     Résolution Vélocité

   (CCD)                  (Séparation)          (Rebond / Friction)
```

b. Processus Détaillé

Étape 1 : Détection

```
List<CollisionInfo> collisions = collisionDetector.DetectCollisions(
   collisionShape,
   physics.Velocity,
   subDeltaTime
);
```

Données retournées :

```
public struct CollisionInfo
{
   public bool hasCollision; // Y a-t-il collision?
   public CollisionShape otherShape; // Avec quel objet?
   public Vector3 normal; // Normale de la surface
   public Vector3 point; // Point de contact
   public float timeOfImpact; // Quand la collision va-t-elle se produire?
   public float penetrationDepth; // Profondeur de pénétration
}
```

Étape 2 : Déplacement jusqu'à la Collision

```
if (collision.timeOfImpact > PhysicsConstants.CCD_TOI_EPSILON)
{
   // Avancer jusqu'au point de collision
   transform.position += physics.Velocity * collision.timeOfImpact * subDeltaTime;
}
```

Étape 3 : Correction de Position (Séparation)

```
// Éloigner l'objet de la surface selon la pénétration
Vector3 separationVector = collision.normal * (collision.penetrationDepth +
SEPARATION_BIASphysics.CorrectPosition(separationVector);
```

Paramètre SEPARATION_BIAS : Petit décalage pour éviter les "jitters" de collision

Étape 4 : Réaction de Vélocité

Type A : Rebond (Elastic)

```
if (physics.Speed >= minBounceVelocity)
{
   physics.Reflect(collision.normal, collision.restitution);
   totalBouncesThisShot++;
}
```

Formule de réflexion :

```
v_reflected = v - 2(v·n)n
v_final = v_reflected × e

Exemple avec v=(1, 1, 0), n=(0, 1, 0), e=0.7 :
v·n = 1
v_reflected = (1, 1, 0) - 2(1)(0, 1, 0) = (1, -1, 0)
v_final = (1, -1, 0) × 0.7 = (0.7, -0.7, 0)
```

Type B : Adhérence (Stick)

```
else
{
   // Vitesse trop faible pour rebondir
   physics.Stop();
   physics.SetGravityEnabled(false);
}
```

Étape 5 : Mise à Jour du Temps Restant

```
// Réduire le temps disponible pour cette itération

remainingTime -= collision.timeOfImpact;

// Continuer la boucle CCD si temps restant significatif

if (remainingTime > PhysicsConstants.CCD_MIN_REMAINING_TIME) continue;
else
break;
```

c. Cas Spéciaux de Collision

Collision avec le Terrain (Plan)

```
Réaction : Rebond partiel selon l'angle d'impact

  -  Impact perpendiculaire (v·n  -1) : rebond maximal
  -  Impact rasant (v·n  0) : friction maximale
```

Collision avec Obstacles (Box/Cylinder)

```
Réaction : Rebond selon la normale calculée à la surface

  -  Normale peut être ajustée selon la courbe de la géométrie
```

Collision Multiple (Plus de 16 itérations)

```
Condition : Si CCD_MAX_ITERATIONS = 16 atteint
Résolution : Forcer l'arrêt ou placer l'objet en surface sûre
Raison : Éviter les boucles infinies sur les arêtes acérées
```

d. Paramètres de Collision Ajustables

Paramètre	Valeur	Impact
restitution	0.7	Contrôle la perte d'énergie lors du rebond
minBouncVelocity	0.2 m/s	Seuil en-dessous duquel le balle ne rebondit plus
linearDrag	0.5 s^{1}	Vitesse de ralentissement dues à la friction
CCD_MAX_ITERATIONS	16	Nombre max de sous-cycles par frame
SEPARATION_BIAS	~0.001	Décalage pour éviter les collisions répétées

e. Exemple de Séquence Complète

Scénario : Balle lancée vers une pente

```
Frame 1 (FixedUpdate - t=0.02s) :
      1. Vélocité initiale : v = (2, 0, 0) m/s
      2. CCD itération 1 :
            - Détecte collision avec plan (terrain)
            - TOI = 0.01s (collision au milieu du frame)
            - Déplace balle jusqu'au point d'impact
            - Calcule normale n = (0, 1, 0)
            - Applique réflexion : v' = (2, 0, 0) × 0.7 = (1.4, 0, 0) 3. Temps
            restant : 0.01s
      4. CCD itération 2 :
            - Aucune collision détectée pour le 0.01s restant
            - Déplace de (1.4, 0, 0) × 0.01 = (0.014, 0, 0)
            - Boucle CCD termine
```

```
Frame 2 (FixedUpdate - t=0.04s) :
      1. Vélocité courante : v = (1.4, 0, 0)
      2. Application drag : v = (1.4, 0, 0) × e^(-0.5×0.02)  (1.385, 0, 0) 3. Pas
      de collision
      4. Déplace de (1.385, 0, 0) × 0.02
      …
```

---

7. Architecture Modulaire

Dépendances des Composants

```
BallController
       Dépend de : CustomPhysics
       Dépend de : SphereCollisionShape
       Dépend de : CollisionDetector
             Dépend de : CollisionShape (base class)
             Utilise : SphereCollisionShape
             Utilise : BoxCollisionShape
             Utilise : CylinderCollisionShape
             Utilise : PlaneCollisionShape

CustomPhysics
       Gère : Vélocité, Accélération
       Applique : Intégration numérique
       Applique : Friction/Drag

CollisionDetector (Singleton)
       Enregistre : Toutes les formes de collision de la scène
```

---

8. Conclusion

Le système minigolf implémente un simulateur physique robuste avec :

- **Physique réaliste** : Lois de Newton, d'intégration Euler semi-implicite, friction
- **Détection de collision avancée** : CCD pour haute vélocité, tri par proximité
- **Formes variées** : Sphère, Box, Cylindre, Plan
- **Réactions physiques** : Rebond avec coefficient de restitution, friction
- **Optimisation** : Séparation statique / dynamique, seuils intelligents
- **Extensibilité** : Architecture modulaire facile à étendre

Ce document sert de référence pour comprendre chaque aspect du jeu et du moteur personnalisé.